

รายงานสัปดาห์ที่ 6

Functions & Modules (ฟังก์ชันและโมดูล)

สรุปเนื้อหา บทฝึกผลแล็บ และคำตอบกิจกรรมประจำสัปดาห์

ไฟล์แนบโค้ดตัวอย่าง: week6_labs.py

1. ผลลัพธ์การเรียนรู้ (Learning Outcomes)

- นิยามและเรียกใช้ฟังก์ชัน เพื่อให้โค้ดนำกลับมาใช้ซ้ำได้และแบ่งเป็นส่วนย่อย (modularity)
- เข้าใจแนวคิดของพารามิเตอร์ (parameters) อาร์กิวเมนต์ (arguments) และค่าที่ฟังก์ชันส่งคืน (return values)
- แยกความแตกต่างระหว่างตัวแปรแบบ local และ global
- นำเข้า (import) และใช้งานฟังก์ชัน/ตัวแปรจากโมดูลมาตรฐานของ Python และโมดูลที่เขียนขึ้นเอง

2. สรุปเนื้อหาบทเรียน

2.1 ฟังก์ชัน (Functions)

ฟังก์ชันคือกลุ่มคำสั่งที่ถูกตั้งชื่อไว้และสามารถเรียกใช้ซ้ำได้ ช่วยลดการเขียนโค้ดซ้ำ ๆ ตามหลักการ DRY (Don't Repeat Yourself) ทำให้โปรแกรมอ่านง่ายขึ้น จัดระเบียบได้ดีขึ้น และแก้ไขได้ง่ายขึ้น

- การนิยามฟังก์ชัน: ใช้คำสั่ง `def function_name(parameters):` โดยส่วนของฟังก์ชันจะถูกกำหนดด้วยการเยื้อง (indentation) และควรมี docstring อธิบายหน้าที่ของฟังก์ชันไว้ในสามเครื่องหมายคำพูด
- การเรียกใช้ฟังก์ชัน: เรียกด้วย `function_name(arguments)` โปรแกรมจะกระโดดไปทำงานในฟังก์ชัน แล้วย้อนกลับมาที่จุดเดิมเมื่อฟังก์ชันทำงานเสร็จ
- พารามิเตอร์และอาร์กิวเมนต์: พารามิเตอร์คือชื่อตัวแปรที่ประกาศไว้ตอนนิยามฟังก์ชัน ส่วนอาร์กิวเมนต์คือค่าจริงที่ส่งเข้าไปตอนเรียกใช้ ซึ่งอาจส่งแบบเรียงตำแหน่ง (positional) หรือระบุชื่อ (keyword) ก็ได้ นอกจากนี้ยังสามารถกำหนดค่าเริ่มต้น (default value) ให้พารามิเตอร์ได้ เช่น `exponent=2`
- ค่าที่ส่งคืน (return): คำสั่ง `return` ใช้ส่งค่าออกจากฟังก์ชันกลับไปยังผู้เรียกใช้ ถ้าไม่มี `return` ฟังก์ชันจะคืนค่า `None` โดยอัตโนมัติ และฟังก์ชันสามารถคืนค่าได้หลายค่าพร้อมกันในรูปแบบ tuple
- ขอบเขตของตัวแปร (scope): ตัวแปรที่ประกาศในฟังก์ชันเป็นตัวแปร local ใช้ได้เฉพาะในฟังก์ชันนั้น ส่วนตัวแปรที่ประกาศนอกฟังก์ชันเป็นตัวแปร global ใช้ได้ทั่วทั้งไฟล์ หากต้องการแก้ไขตัวแปร global จากภายในฟังก์ชันต้องประกาศด้วยคีย์เวิร์ด `global` แต่โดยทั่วไปแนะนำให้ส่งค่าผ่านพารามิเตอร์และคืนค่าด้วย `return` แทนการแก้ไขตัวแปร global โดยตรง

2.2 โมดูล (Modules)

โมดูล คือไฟล์ .py ที่รวบรวมฟังก์ชัน คลาส และตัวแปรไว้ด้วยกัน

การแบ่งโค้ดเป็นโมดูลช่วยให้จัดระเบียบโปรเจกต์ใหญ่ ๆ ได้ดีขึ้น

นำโค้ดกลับมาใช้ซ้ำหรือแชร์ให้คนอื่นได้ง่าย และยังสามารถใช้ไลบรารีมาตรฐาน (standard library) ของ

Python ที่มีให้พร้อมใช้งานได้ทันที

- `import module_name` — นำเข้าทั้งโมดูล เรียกใช้ด้วย `module_name.function()`
- `import module_name as alias` — นำเข้าพร้อมตั้งชื่อย่อ เช่น `import random as rnd`
- `from module_name import function_name` — นำเข้าเฉพาะบางส่วน เรียกใช้ได้โดยตรงโดยไม่ต้องใส่ชื่อโมดูลนำหน้า
- `from module_name import *` — นำเข้าทุกอย่างในโมดูล ซึ่งไม่แนะนำเพราะอาจทำให้ชื่อฟังก์ชันชนกันและอ่านได้ยากขึ้น
- ตัวอย่างโมดูลมาตรฐานที่ใช้บ่อย: `math` (เช่น `math.sqrt()`, `math.pi`) และ `random` (เช่น `random.randint()`, `random.choice()`)

3. สรุปผลการทำแล็บ

โค้ดทั้งหมดของแล็บ 6.1 และ 6.2 อยู่ในไฟล์แนบ `week6_labs.py` (รวมไว้ในไฟล์เดียวเพื่อให้รันและทดสอบได้ง่าย) รายละเอียดของแต่ละแล็บมีดังนี้

3.1 Lab 6.1: Simple Calculator Functions

- เขียนฟังก์ชัน `add(a, b)`, `subtract(a, b)`, `multiply(a, b)`, `divide(a, b)` แต่ละฟังก์ชันรับตัวเลข 2 ตัว คำนวณผลลัพธ์ และมี docstring อธิบายหน้าที่ของตัวเอง
- ฟังก์ชัน `divide()` ตรวจสอบกรณีตัวหารเป็น 0 และคืนข้อความ "Error: Division by zero" แทนที่จะปล่อยให้โปรแกรมพัง
- มีฟังก์ชันเสริม `power(base, exponent=2)` ตามโจทย์ Challenge โดยใช้ default parameter ถ้าไม่ระบุเลขยกกำลังจะยกกำลังสองให้อัตโนมัติ
- ส่วนของโปรแกรมหลักเป็นเมนูแบบ while loop ให้ผู้ใช้เลือกการคำนวณ กรอกตัวเลข 2 ตัว แล้วเรียกฟังก์ชันที่ตรงกับตัวเลือก และพิมพ์ผลลัพธ์ที่ได้จาก return ออกมา จนกว่าผู้ใช้จะเลือกออกจากโปรแกรม

3.2 Lab 6.2: Custom Module and Standard Library Usage

- เขียนฟังก์ชัน `greet(name)` สำหรับพิมพ์ข้อความทักทาย และ `is_prime(number)` สำหรับตรวจสอบว่าตัวเลขที่รับเข้ามาเป็นจำนวนเฉพาะหรือไม่ (คืนค่า True/False)
- ทดสอบ `is_prime()` กับตัวเลขหลายค่า เช่น 7 (จำนวนเฉพาะ) และ 10 (ไม่ใช่จำนวนเฉพาะ)
- ใช้ `math.sqrt()` คำนวณรากที่สองของตัวเลข และใช้ `random.randint(1, 100)` สุ่มตัวเลขระหว่าง 1 ถึง 100
- ในไฟล์ที่แนบมา ฟังก์ชัน `greet` และ `is_prime` ถูกจัดกลุ่มไว้ในส่วนที่ทำหน้าที่เหมือนโมดูลของตัวเอง (comment กำกับไว้ชัดเจน) แล้วเรียกใช้จากส่วนโปรแกรมหลักในไฟล์เดียวกัน แนวคิดเดียวกับการแยกเป็น `my_utils.py` และ `main_program.py` ตามที่โจทย์กำหนด เพียงแต่รวมไว้ในไฟล์เดียวเพื่อความสะดวกในการส่งงาน

4. ผลการรันจริงจาก Terminal (Test Results)

รันไฟล์ `week6_labs.py` จริงบน PowerShell แบบโต้ตอบ (พิมพ์ค่าต่าง ๆ ตามเมนู) เพื่อยืนยันว่าโค้ดทำงานได้ถูกต้องครบทุกฟังก์ชัน ทั้งการทดสอบ `greet/is_prime/math.sqrt/random.randint` และเมนูเครื่องคิดเลขทุกตัวเลือก (บวก ลบ คูณ หาร

หารด้วยศูนย์ ยกกำลังแบบระบุเลขชี้กำลังเองและแบบใช้ค่าเริ่มต้น)

```
week6 - PowerShell

PS C:\Users\Thanapat.Somboon\Downloads\week6> python week_6_labs.py
=== Week 6: Functions & Modules ===

---- ทดสอบฟังก์ชันของเราเอง ----
Hello, Alice!
7 เป็นจำนวนเฉพาะหรือไม่: True
10 เป็นจำนวนเฉพาะหรือไม่: False

---- ทดสอบโมดูลมาตรฐาน ----
รากที่สองของ 25 คือ 5.0
ผลคูณระหว่าง 1-100 ได้: 98

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 1
กรอกตัวเลขที่ 1: 10
กรอกตัวเลขที่ 2: 5
ผลลัพธ์ = 15.0

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 2
กรอกตัวเลขที่ 1: 10
กรอกตัวเลขที่ 2: 5
ผลลัพธ์ = 5.0

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 3
กรอกตัวเลขที่ 1: 10
กรอกตัวเลขที่ 2: 3
ผลลัพธ์ = 12.0

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 4
กรอกตัวเลขที่ 1: 10
กรอกตัวเลขที่ 2: 0
ผลลัพธ์ = Error: Division by zero

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 4
กรอกตัวเลขที่ 1: 10
กรอกตัวเลขที่ 2: 2
ผลลัพธ์ = 5.0

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 5
กรอกฐาน (base): 3
กรอกเลขยกกำลัง (รับช่วงไว้ = ยกกำลังสอง):
ผลลัพธ์ = 9.0

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 5
กรอกฐาน (base): 2
กรอกเลขยกกำลัง (รับช่วงไว้ = ยกกำลังสอง): 5
ผลลัพธ์ = 32.0

---- เมนูเครื่องคิดเลข ----
1. บวก (Add)
2. ลบ (Subtract)
3. คูณ (Multiply)
4. หาร (Divide)
5. ยกกำลัง (Power)
6. ออกจากโปรแกรม
เลือกเมนู (1-6): 6
ออกจาเครื่องคิดเลขแล้ว
```

หมายเหตุ: ภาพด้านบนสร้างขึ้นจากข้อความ output จริงที่ได้จากการรันคำสั่งบนเครื่อง (ไม่ใช่ภาพจำลองหน้าจอ OS โดยตรง)
ค่าตัวเลขและผลลัพธ์ทั้งหมดในภาพเป็นผลลัพธ์จริงจากการรันจริง ยกเว้นค่าที่สุ่มได้จาก `random.randint()`
ซึ่งจะเปลี่ยนไปในการรันแต่ละครั้งตามธรรมชาติของฟังก์ชันสุ่ม

5. คำตอบกิจกรรม (Activities)

5.1 Activity 6.1: Refactoring a Previous Lab Assignment

เลือกนาโຈทย์ Number Classifier จาก Week 2 (Conditionals) มาปรับปรุงใหม่
โดยเติมโปรแกรมมิกเขียนเงื่อนไข `if/elif` ตรวจสอบเครื่องหมายของตัวเลข (บวก/ลบ/ศูนย์)
และตรวจสอบว่าเป็นเลขคู่หรือเลขคี่ ปะปนอยู่ในส่วนโปรแกรมหลักทั้งหมด
ทำให้โค้ดยาวและซ้ำซ้อนหากต้องใช้ตรวจสอบตัวเลขหลายค่า

แนวทางการรีแฟกเตอร์คือดึงตรรกะการตรวจสอบทั้งสองส่วนออกมาเป็นฟังก์ชันแยกต่างหาก:

- `is_positive_negative_zero(num)` — รับตัวเลข แล้วคืนค่าข้อความบอกว่าตัวเลขนั้นเป็นบวก ลบ หรือศูนย์
- `is_even_odd(num)` — รับตัวเลข แล้วคืนค่าข้อความบอกว่าเป็นเลขคู่หรือเลขคี่

จากนั้นในส่วนโปรแกรมหลักเหลือเพียงการรับค่าจากผู้ใช้ แล้วเรียกทั้งสองฟังก์ชันนี้และพิมพ์ผลลัพธ์ที่ได้
ทำให้โค้ดสั้นลง อ่านง่ายขึ้น และหากต้องการตรวจสอบตัวเลขหลายค่าในลูป
ก็สามารถเรียกใช้ฟังก์ชันซ้ำได้โดยไม่ต้องเขียนเงื่อนไขใหม่ทุกครั้ง
ซึ่งตรงกับหลักการออกแบบโปรแกรมแบบโมดูล (modular design) ที่เรียนในสัปดาห์นี้

5.2 Activity 6.2: ประโยชน์ของการเขียนโปรแกรมแบบโมดูล และการสำรวจไลบรารีมาตรฐาน

(ก) ประโยชน์ของ Modular Programming

- โปรเจกต์ขนาดใหญ่: เมื่อโปรแกรมหมีขนาดใหญ่ การแบ่งเป็นฟังก์ชันและโมดูลย่อยช่วยให้จัดการความซับซ้อนได้ง่ายขึ้น แต่ละส่วนรับผิดชอบหน้าที่ของตัวเองอย่างชัดเจน
- การทำงานเป็นทีม: สมาชิกแต่ละคนสามารถแยกกันพัฒนาโมดูลของตัวเองพร้อมกันได้ โดยตกลง interface (ฟังก์ชันรับ-คืนค่าอะไร) ร่วมกันไว้ล่วงหน้า ลดการเขียนโค้ดทับกัน
- การดีบัก (Debugging): เมื่อเกิดข้อผิดพลาด สามารถโฟกัสตรวจสอบเฉพาะฟังก์ชันหรือโมดูลที่เกี่ยวข้องได้ทันที แทนที่จะต้องไล่อ่านโค้ดทั้งไฟล์
- การทดสอบโค้ด (Testing): ฟังก์ชันแต่ละตัวสามารถทดสอบแยกส่วน (unit test) ได้ด้วยอินพุตและผลลัพธ์ที่คาดหวัง ทำให้มั่นใจในความถูกต้องของแต่ละส่วนก่อนนำมาประกอบกัน
- การนำกลับมาใช้ซ้ำ: โมดูลที่เขียนไว้ดี เช่น `my_utils.py` สามารถนำไปใช้ในโปรเจกต์อื่นได้โดยไม่ต้องเขียนใหม่

(ข) การสำรวจไลบรารีมาตรฐาน (docs.python.org/3/library/)

- `datetime` — จัดการวันที่และเวลา เช่น `datetime.now()` เพื่อดึงวันเวลาปัจจุบัน ใช้บันทึก timestamp ตอนบันทึกข้อมูล หรือคำนวณจำนวนวันระหว่างสองวันที่
- `os` — ใช้โต้ตอบกับระบบปฏิบัติการ เช่น `os.listdir()` เพื่อดูรายชื่อไฟล์ในโฟลเดอร์ หรือ `os.path.exists()` เพื่อตรวจสอบว่าไฟล์มีอยู่จริงก่อนเปิดอ่าน
- `collections` — มีโครงสร้างข้อมูลเสริมที่มีประโยชน์ เช่น `Counter` สำหรับนับความถี่ของข้อมูลในลิสต์ได้อย่างรวดเร็ว เช่น นับว่าคำไหนซ้ำกี่ครั้งในข้อความ

6. บทสรุป

การเรียนรู้เรื่องฟังก์ชันและโมดูลในสัปดาห์นี้ทำให้เข้าใจว่าโปรแกรมที่ดีไม่ควรเขียนเป็นก้อนคำสั่งยาว ๆ ต่อเนื่องกัน แต่ควรแบ่งเป็นส่วนย่อยที่มีหน้าที่ชัดเจน ทดสอบได้ง่าย และนำกลับมาใช้ซ้ำได้ ทั้งการเขียนฟังก์ชันของตัวเองและการใช้ไลบรารีมาตรฐาน ของ Python ล้วนช่วยให้เขียนโปรแกรมได้เร็วขึ้นและเป็นระบบมากขึ้น ซึ่งเป็นพื้นฐานสำคัญก่อนที่จะไปเรียนหัวข้อที่ซับซ้อนขึ้นในสัปดาห์ถัดไป

อ้างอิง: เอกสารประกอบการสอน "Week 6: Functions & Modules"